

2547160_Lab03_ML

June 24, 2026

1 Lab 3

1.1 Part A: Data Collection and Preprocessing

1. Load the dataset using Pandas

```
[122]: import pandas as pd
```

2. Display the first 5 rows

```
[123]: df = pd.read_excel("../data/Department Awareness Survey (Responses).xlsx")
df.head()
```

```
[123]:
```

	Timestamp	Registration Number	\
0	2026-06-22 08:49:31.362	2547123	
1	2026-06-22 08:49:50.520	2547122	
2	2026-06-22 08:50:44.521	2547101	
3	2026-06-22 08:51:00.666	2547156	
4	2026-06-22 08:51:42.357	2547148	

	Email	\
0	jiyaelza.jabi@mca.christuniversity.in	
1	jinishaleema.rosario@mca.christuniversity.in	
2	rajeev.chandar@mca.christuniversity.in	
3	sounak.chakraborty@mca.christuniversity.in	
4	samar.subhash@mca.christuniversity.in	

	Job role that you are interested in\n	\
0	Software Engineer	
1	Data Engineer	
2	Software Engineer	
3	Data Scientist	
4	Software Engineer	

What is the minimum salary of students placed through campus (In LPA..respond as a number) \

0	4
1	7,00,000
2	6

3	4.5
4	8

What is the maximum salary of students placed through campus (In LPA..respond as a number) \

0	Option 1
1	Option 1
2	17
3	12
4	12

What is the median salary of students placed through campus (In LPA..respond as a number) \

0	9
1	4,50,000
2	8
3	6.9
4	10

Which is the highest paying company \

0	Deolite
1	Deolite
2	DE Shaw
3	DE Shaw
4	EY

Rate your contribution towards extra curricular activities \

0	2
1	4
2	5
3	3
4	5

Rate your technical competencies What are your package expectations (LPA) \

0	2	7
1	3	6,00,000
2	5	13
3	4	7
4	3	10

your CIA % of last semester your GPA of last semester \

0	78	3.24
1	64.78	3.2
2	80	3.6
3	70	3.2
4	68	3.4

	Your maximum attendance % till last semester	Internships	Interests
0		Option 1	Industry
1		Option 1	Industry
2		Option 1	Industry
3		92	Industry
4		85	Industry

3. Check dataset dimensions

```
[124]: df.shape
```

```
[124]: (54, 15)
```

4. Improving Data Quality and identifying missing values
5. Remove or handle null values
6. Convert required columns into numerical datatype

```
[125]: df.isna().sum()
```

```
[125]: Timestamp
0
Registration Number
0
Email
0
Job role that you are interested in\n
0
What is the minimum salary of students placed through campus (In LPA..respond as
a number)    0
What is the maximum salary of students placed through campus (In LPA..respond as
a number)    0
What is the median salary of students placed through campus (In LPA..respond as
a number)    0
Which is the highest paying company
0
Rate your contribution towards extra curricular activities
0
Rate your technical competencies
0
What are your package expectations (LPA)
0
your CIA % of last semester
0
your GPA of last semester
0
Your maximum attendance % till last semester
0
Internships Interests
```

```
0
dtype: int64
```

```
[126]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54 entries, 0 to 53
Data columns (total 15 columns):
 #   Column
Non-Null Count  Dtype
---  -
0    Timestamp
54 non-null     datetime64[ns]
1    Registration Number
54 non-null     object
2    Email
54 non-null     object
3    Job role that you are interested in
54 non-null     object
4    What is the minimum salary of students placed through campus (In
LPA..respond as a number) 54 non-null     object
5    What is the maximum salary of students placed through campus (In
LPA..respond as a number) 54 non-null     object
6    What is the median salary of students placed through campus (In
LPA..respond as a number) 54 non-null     object
7    Which is the highest paying company
54 non-null     object
8    Rate your contribution towards extra curricular activities
54 non-null     int64
9    Rate your technical competencies
54 non-null     int64
10   What are your package expectations (LPA)
54 non-null     object
11   your CIA % of last semester
54 non-null     object
12   your GPA of last semester
54 non-null     object
13   Your maximum attendance % till last semester
54 non-null     object
14   Internships Interests
54 non-null     object
dtypes: datetime64[ns](1), int64(2), object(12)
memory usage: 6.5+ KB
```

```
[127]: df.rename(columns={
        'Timestamp': 'timestamp',
        'Registration Number': 'reg_no',
```

```

    'Email': 'email',
    'Job role that you are interested in\n': 'job_role',
    'What is the minimum salary of students placed through campus (In LPA..
↳respond as a number)': 'min_salary_lpa',
    'What is the maximum salary of students placed through campus (In LPA..
↳respond as a number)': 'max_salary_lpa',
    'What is the median salary of students placed through campus (In LPA..
↳respond as a number)': 'median_salary_lpa',
    'Which is the highest paying company': 'top_company',
    'Rate your contribution towards extra curricular activities': '
↳extracurricular_rating',
    'Rate your technical competencies': 'tech_rating',
    'What are your package expectations (LPA)': 'package_expectation_lpa',
    'your CIA % of last semester': 'cia_pct',
    'your GPA of last semester': 'gpa',
    'Your maximum attendance % till last semester': 'attendance_pct',
    'Internships Interests': 'internship_interest'
}, inplace=True)

df.columns.tolist()

```

```

[127]: ['timestamp',
        'Registration Number ',
        'email',
        'job_role',
        'min_salary_lpa',
        'max_salary_lpa',
        'median_salary_lpa',
        'top_company',
        'extracurricular_rating',
        'tech_rating',
        'package_expectation_lpa',
        'your CIA % of last semester ',
        'gpa',
        'attendance_pct',
        'internship_interest']

```

```

[128]: df.columns = df.columns.str.strip()

```

```

[129]: df.rename(columns={
        'Registration Number': 'reg_no',
        'your CIA % of last semester': 'cia_pct'
    }, inplace=True)

df.columns.tolist()

```

```
[129]: ['timestamp',
        'reg_no',
        'email',
        'job_role',
        'min_salary_lpa',
        'max_salary_lpa',
        'median_salary_lpa',
        'top_company',
        'extracurricular_rating',
        'tech_rating',
        'package_expectation_lpa',
        'cia_pct',
        'gpa',
        'attendance_pct',
        'internship_interest']
```

Target Columns: CIA, GPA, Attendance

```
[130]: print("CIA % ")
        print(df['cia_pct'].value_counts())
```

```
CIA %
cia_pct
70      12
80       6
78       4
0.7      3
71       2
64.78    1
0.88     1
68       1
83       1
64.67    1
89       1
0.7111   1
66       1
67       1
84       1
3.45     1
60       1
85       1
75       1
0.77     1
-        1
70.32    1
0.8      1
0.75     1
77       1
```

```

72      1
4       1
71.5    1
76      1
79.33   1
86      1
0.63    1
Name: count, dtype: int64

```

```

[131]: import numpy as np
def clean_cia(val):
    val = str(val).strip()
    if val == '-':
        return np.nan
    val = float(val)
    if val <= 1:
        return val * 100
    if val < 10:
        return np.nan
    return val

```

```

[132]: df['cia_pct'] = df['cia_pct'].apply(clean_cia)

```

```

[133]: print("\nGPA: ")
print(df['gpa'].value_counts())

```

```

GPA:
gpa
3.6      8
3.4      7
3.5      4
3.2      2
3.7      2
3.3      2
2.9      2
3.8      2
4         2
3.46     2
3         2
3.24     1
3.89     1
10       1
3.75     1
3.37     1
3.67     1
3.48     1
3.86     1

```

```

3.64    1
90      1
na      1
3.51    1
3.29    1
2.93    1
3.11    1
3.27    1
3.77    1
3.74    1
7.8     1
Name: count, dtype: int64

```

```

[134]: def clean_gpa(val):
        val = str(val).strip().lower()
        if val == 'na':
            return np.nan
        val = float(val)
        if val > 4:
            return np.nan
        return val

```

```

[135]: df['gpa'] = df['gpa'].apply(clean_gpa)

```

```

[136]: print("\nAttendance")
        print(df['attendance_pct'].value_counts())

```

```

Attendance
attendance_pct
Option 1    6
95          5
89          4
85          4
90          3
92          3
100         3
97          2
86          2
98          2
0.9425      1
91          1
0.87        1
0.99        1
0.986       1
93          1
92.08       1
99          1

```



```

0.97      1
96        1
94        1
92.83     1
87        1
98.03     1
0.98      1
0.9       1
97.99     1
88        1
99.34     1
98.71     1
Name: count, dtype: int64

```

```

[137]: def clean_attendance(val):
        val = str(val).strip()
        if val.lower().startswith('option'): # "Option 1" etc
            return np.nan
        val = float(val)
        if val <= 1: # decimal like 0.9425
            return val * 100
        return val

```

```

[138]: df['attendance_pct'] = df['attendance_pct'].apply(clean_attendance)

```

```

[139]: print(df[['cia_pct', 'gpa', 'attendance_pct']].isna().sum())

```

```

cia_pct      3
gpa          4
attendance_pct 6
dtype: int64

```

```

[140]: print(df[['cia_pct', 'gpa', 'attendance_pct']].isna().sum()/df.shape[0]*100)

```

```

cia_pct      5.555556
gpa          7.407407
attendance_pct 11.111111
dtype: float64

```

The value that were replaced with Nan was uninterpretable and cannot be confidently reconstructed. Removing the rows, risks removal of important data in the other columns and since these are target columns themselves, list wise deletion can't be performed either. Therefore, imputing with mean/median would be better suited

```

[141]: df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54 entries, 0 to 53
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype

```

```

---  -----
0    timestamp                54 non-null    datetime64[ns]
1    reg_no                   54 non-null    object
2    email                    54 non-null    object
3    job_role                  54 non-null    object
4    min_salary_lpa           54 non-null    object
5    max_salary_lpa           54 non-null    object
6    median_salary_lpa        54 non-null    object
7    top_company               54 non-null    object
8    extracurricular_rating    54 non-null    int64
9    tech_rating              54 non-null    int64
10   package_expectation_lpa   54 non-null    object
11   cia_pct                   51 non-null    float64
12   gpa                       50 non-null    float64
13   attendance_pct           48 non-null    float64
14   internship_interest       54 non-null    object
dtypes: datetime64[ns](1), float64(3), int64(2), object(9)
memory usage: 6.5+ KB

```

```

[142]: import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

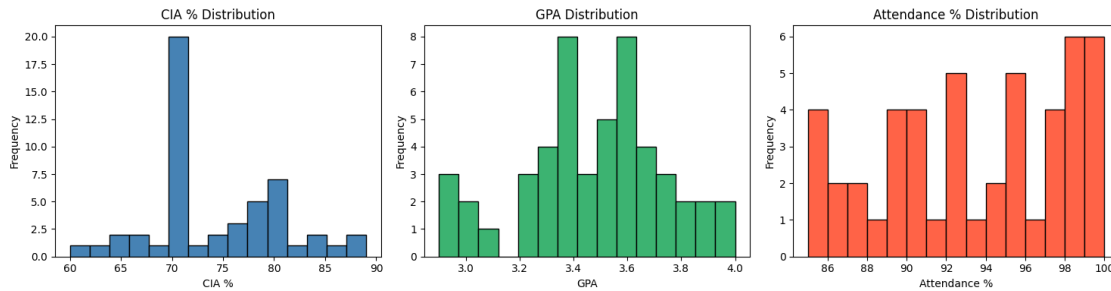
df['cia_pct'].plot(kind='hist', ax=axes[0], bins=15, color='steelblue',
    ↪edgecolor='black')
axes[0].set_title('CIA % Distribution')
axes[0].set_xlabel('CIA %')

df['gpa'].plot(kind='hist', ax=axes[1], bins=15, color='mediumseagreen',
    ↪edgecolor='black')
axes[1].set_title('GPA Distribution')
axes[1].set_xlabel('GPA')

df['attendance_pct'].plot(kind='hist', ax=axes[2], bins=15, color='tomato',
    ↪edgecolor='black')
axes[2].set_title('Attendance % Distribution')
axes[2].set_xlabel('Attendance %')

plt.tight_layout()
plt.show()

```



```
[143]: print("Percentage: \n")
print(f"Mean: {df['cia_pct'].mean():.2f}")
print(f"Median: {df['cia_pct'].median():.2f}")

print("\nGPA\n")
print(f"Mean: {df['gpa'].mean():.2f}")
print(f"Median: {df['gpa'].median():.2f}")

print("\nAttendance %: \n")
print(f"Mean: {df['attendance_pct'].mean():.2f}")
print(f"Median: {df['attendance_pct'].median():.2f}")
```

Percentage:

Mean: 74.05
Median: 71.11

GPA

Mean: 3.48
Median: 3.50

Attendance %:

Mean: 93.12
Median: 93.50

CIA % — Mean: 74.05 | Median: 71.11 Gap of ~3 points suggests mild right skew, a few high scorers pulling the mean up. Median is safer here.

GPA — Mean: 3.48 | Median: 3.50 Mean and median are almost identical, distribution is symmetric. Either works, using mean.

Attendance % — Mean: 93.12 | Median: 93.50 Very close values, slight left skew from a few low attenders pulling mean down. Median is safer here.

```
[144]: df['cia_pct'].fillna(df['cia_pct'].median(), inplace=True)
df['gpa'].fillna(df['gpa'].mean(), inplace=True)
df['attendance_pct'].fillna(df['attendance_pct'].median(), inplace=True)

print(df[['cia_pct', 'gpa', 'attendance_pct']].isna().sum())
```

```
cia_pct      0
gpa          0
attendance_pct 0
dtype: int64
```

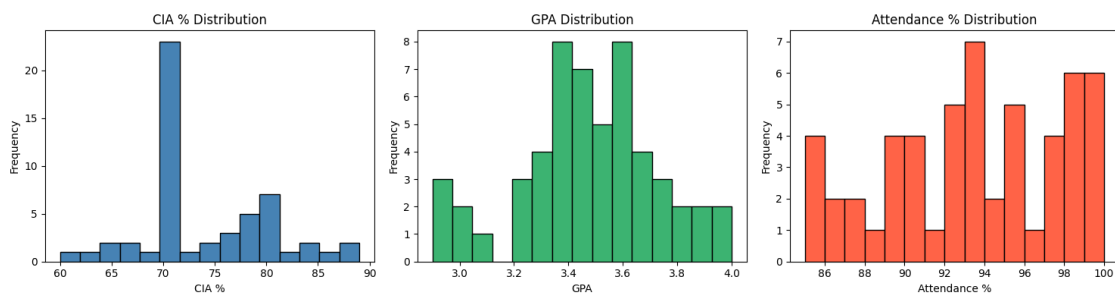
```
[145]: fig, axes = plt.subplots(1, 3, figsize=(15, 4))

df['cia_pct'].plot(kind='hist', ax=axes[0], bins=15, color='steelblue',
                  edgecolor='black')
axes[0].set_title('CIA % Distribution')
axes[0].set_xlabel('CIA %')

df['gpa'].plot(kind='hist', ax=axes[1], bins=15, color='mediumseagreen',
               edgecolor='black')
axes[1].set_title('GPA Distribution')
axes[1].set_xlabel('GPA')

df['attendance_pct'].plot(kind='hist', ax=axes[2], bins=15, color='tomato',
                        edgecolor='black')
axes[2].set_title('Attendance % Distribution')
axes[2].set_xlabel('Attendance %')

plt.tight_layout()
plt.show()
```



```
[146]: print("Percentage: \n")
print(f"Mean:    {df['cia_pct'].mean():.2f}")
print(f"Median: {df['cia_pct'].median():.2f}")

print("\nGPA\n")
```

```

print(f"Mean: {df['gpa'].mean():.2f}")
print(f"Median: {df['gpa'].median():.2f}")

print("\nAttendance %: \n")
print(f"Mean: {df['attendance_pct'].mean():.2f}")
print(f"Median: {df['attendance_pct'].median():.2f}")

```

Percentage:

Mean: 73.89
Median: 71.11

GPA

Mean: 3.48
Median: 3.48

Attendance %:

Mean: 93.16
Median: 93.50

Distribution after imputation remains the same

```

[147]: remaining = ['min_salary_lpa', 'max_salary_lpa', 'median_salary_lpa',
                    'package_expectation_lpa', 'top_company', 'job_role',
                    'internship_interest']

for col in remaining:
    print(f"\n=== {col} ===")
    print(df[col].value_counts())

```

=== min_salary_lpa ===

min_salary_lpa

5	8
4.5	6
4	6
6	4
3	3
4,00,000	2
3.3	2
300000	2
7	2
4 LPA	2
7,00,000	1
3,00,000	1
3.5	1
5,00,000.00	1

3.4	1
8	1
6lpa	1
2.7	1
12LPA	1
4 lpa	1
1500000	1
2	1
12	1
4.5 LPA	1
10lpa	1
6 LPA	1
20000	1

Name: count, dtype: int64

```
=== max_salary_lpa ===
```

max_salary_lpa	
12	23
Option 1	9
24	3
12 LPA	3
25	2
17	1
8	1
22,00,000	1
12lpa	1
12,00,000.00	1
10,00,000	1
22 lpa	1
1200000	1
1300000	1
15	1
14	1
20	1
100lpa	1
80000	1

Name: count, dtype: int64

```
=== median_salary_lpa ===
```

median_salary_lpa	
7	12
6	8
8	4
9	2
4,50,000	2
6.9	2
10	2
5	2

6.3	2
7 LPA	2
8,50,000.00	1
12	1
6,00,000	1
6.9LPA	1
700000	1
450000	1
15	1
7lpa	1
7,.5	1
1100000	1
5 LPA	1
9.5	1
500000	1
20lpa	1
4500000	1
6 LPA	1

Name: count, dtype: int64

```
=== package_expectation_lpa ===
```

package_expectation_lpa	
10	8
8	8
7	4
12	4
13	3
6	3
6,00,000	3
15	2
5	2
7 LPA	2
9	1
10 LPA	1
8,00,000	1
700000	1
8+	1
8 to 12 lpa	1
10LPA	1
1500000	1
7-9	1
8-9	1
5 to 9 LPA	1
1200000	1
2cr	1
14	1
60	1

Name: count, dtype: int64

```

=== top_company ===
top_company
DE Shaw      38
Capgemini    7
Deolite      6
EY           3
Name: count, dtype: int64

```

```

=== job_role ===
job_role
Software Engineer    28
Business Analyst    11
Data Engineer       8
Data Scientist       7
Name: count, dtype: int64

```

```

=== internship_interest ===
internship_interest
Industry    53
Research     1
Name: count, dtype: int64

```

```

[148]: import re

def clean_salary(val):
    val = str(val).strip().lower()

    # unusable
    if val.startswith('option'):
        return np.nan
    if val == 'nan':
        return np.nan

    # crore handling
    if 'cr' in val:
        val = val.replace('cr', '').replace('crore', '').strip()
        return float(val) * 100

    # strip "lpa" text
    val = val.replace('lpa', '').strip()

    # range handling like "3-4" or "3 - 4"
    if '-' in val:
        parts = val.split('-')
        try:
            return (float(parts[0].strip()) + float(parts[1].strip())) / 2

```



```

        except:
            return np.nan

        # strip commas (for 7,00,000 style)
        val = val.replace(',', '').strip()

        # convert to float
        try:
            val = float(val)
        except:
            return np.nan

        if val >= 10000:
            return val / 100000

        return val

df['min_salary_lpa'] = df['min_salary_lpa'].apply(clean_salary)
df['max_salary_lpa'] = df['max_salary_lpa'].apply(clean_salary)
df['median_salary_lpa'] = df['median_salary_lpa'].apply(clean_salary)
df['package_expectation_lpa'] = df['package_expectation_lpa'].
    ↪ apply(clean_salary)

for col in ['min_salary_lpa', 'max_salary_lpa', 'median_salary_lpa',
    ↪ 'package_expectation_lpa']:
    print(f"{col}: \n {df[col].isna().sum()} NaNs")
    print(df[col].describe())
    print()

```

```

min_salary_lpa:
  0 NaNs
count    54.000000
mean      5.016667
std       2.512816
min       0.200000
25%      4.000000
50%      4.500000
75%      5.750000
max      15.000000
Name: min_salary_lpa, dtype: float64

```

```

max_salary_lpa:
  9 NaNs
count    45.000000
mean     15.817778
std      13.739910
min       0.800000

```

```

25%      12.000000
50%      12.000000
75%      14.000000
max       100.000000
Name: max_salary_lpa, dtype: float64

```

```

median_salary_lpa:
  0 NaNs
count      54.000000
mean       8.098148
std        5.725464
min        4.500000
25%        6.000000
50%        7.000000
75%        8.000000
max        45.000000
Name: median_salary_lpa, dtype: float64

```

```

package_expectation_lpa:
  3 NaNs
count      51.000000
mean      13.931373
std       27.641277
min        5.000000
25%        7.000000
50%        8.500000
75%       12.000000
max       200.000000
Name: package_expectation_lpa, dtype: float64

```

```

[149]: for col in remaining:
        print(f"\n=== {col} ===")
        print(df[col].value_counts())

```

```

=== min_salary_lpa ===
min_salary_lpa
4.0      11
5.0       9
4.5       7
6.0       6
3.0       6
7.0       3
12.0      2
3.3       2
8.0       1
3.4       1

```

```

3.5      1
2.7      1
15.0     1
2.0      1
10.0     1
0.2      1
Name: count, dtype: int64

```

```

=== max_salary_lpa ===
max_salary_lpa
12.0     29
24.0      3
22.0      2
25.0      2
17.0      1
10.0      1
8.0       1
13.0      1
15.0      1
14.0      1
20.0      1
100.0     1
0.8       1
Name: count, dtype: int64

```

```

=== median_salary_lpa ===
median_salary_lpa
7.0      16
6.0      10
8.0       4
5.0       4
4.5       3
6.9       3
9.0       2
10.0      2
6.3       2
8.5       1
12.0      1
15.0      1
7.5       1
11.0      1
9.5       1
20.0      1
45.0      1
Name: count, dtype: int64

```

```

=== package_expectation_lpa ===
package_expectation_lpa

```

```

10.0    10
8.0     10
7.0     7
6.0     6
12.0    5
15.0    3
13.0    3
5.0     2
9.0     1
8.5     1
200.0   1
14.0    1
60.0    1
Name: count, dtype: int64

```

```

=== top_company ===
top_company
DE Shaw    38
Capgemini  7
Deolite    6
EY         3
Name: count, dtype: int64

```

```

=== job_role ===
job_role
Software Engineer    28
Business Analyst     11
Data Engineer        8
Data Scientist       7
Name: count, dtype: int64

```

```

=== internship_interest ===
internship_interest
Industry    53
Research    1
Name: count, dtype: int64

```

```

[150]: fig, axes = plt.subplots(1, 3, figsize=(15, 4))

df['min_salary_lpa'].plot(kind='hist', ax=axes[0], bins=15, color='steelblue',
    ↪edgecolor='black')
axes[0].set_title('Min Salary LPA Distribution')
axes[0].set_xlabel('Min Salary (LPA)')

df['max_salary_lpa'].plot(kind='hist', ax=axes[1], bins=15,
    ↪color='mediumseagreen', edgecolor='black')
axes[1].set_title('Max Salary LPA Distribution')

```

```

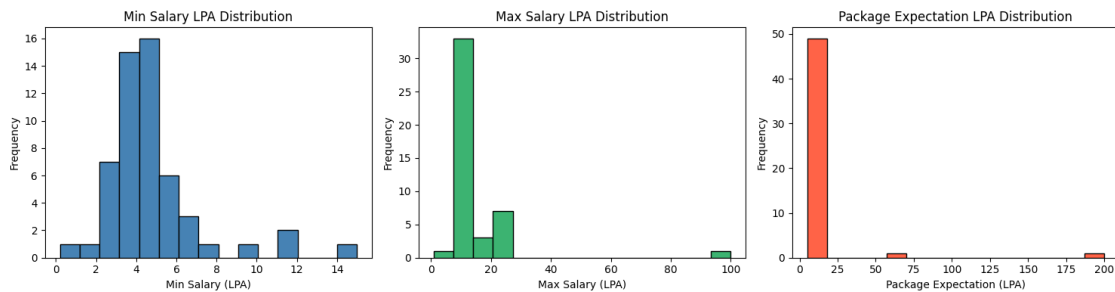
axes[1].set_xlabel('Max Salary (LPA)')

df['package_expectation_lpa'].plot(kind='hist', ax=axes[2], bins=15,
    color='tomato', edgecolor='black')
axes[2].set_title('Package Expectation LPA Distribution')
axes[2].set_xlabel('Package Expectation (LPA)')

plt.tight_layout()
plt.show()

for col in ['min_salary_lpa', 'max_salary_lpa', 'package_expectation_lpa']:
    print(f"\n=== {col} ===")
    print(f"Mean: {df[col].mean():.2f}")
    print(f"Median: {df[col].median():.2f}")

```



```
=== min_salary_lpa ===
```

```
Mean: 5.02
```

```
Median: 4.50
```

```
=== max_salary_lpa ===
```

```
Mean: 15.82
```

```
Median: 12.00
```

```
=== package_expectation_lpa ===
```

```
Mean: 13.93
```

```
Median: 8.50
```

No imputation for Median

```

[151]: df['max_salary_lpa'].fillna(df['max_salary_lpa'].median(), inplace=True)
df['package_expectation_lpa'].fillna(df['package_expectation_lpa'].median(),
    inplace=True)

print(df.isna().sum())

```

timestamp

0

```

reg_no            0
email             0
job_role          0
min_salary_lpa    0
max_salary_lpa    0
median_salary_lpa 0
top_company       0
extracurricular_rating 0
tech_rating       0
package_expectation_lpa 0
cia_pct           0
gpa               0
attendance_pct     0
internship_interest 0
dtype: int64

```

7. Remove duplicate records if present

```
[152]: print(f"Duplicates found: {df.duplicated().sum()}")
```

Duplicates found: 0

8. Generate statistical summary

```
[153]: df.describe()
```

```
[153]:
```

	timestamp	min_salary_lpa	max_salary_lpa	\
count	54	54.000000	54.000000	
mean	2026-06-22 08:53:31.366500096	5.016667	15.181481	
min	2026-06-22 08:49:31.362000	0.200000	0.800000	
25%	2026-06-22 08:52:20.128749824	4.000000	12.000000	
50%	2026-06-22 08:53:30.697499904	4.500000	12.000000	
75%	2026-06-22 08:54:18.161750016	5.750000	12.000000	
max	2026-06-22 08:57:57.108000	15.000000	100.000000	
std	NaN	2.512816	12.601184	

	median_salary_lpa	extracurricular_rating	tech_rating	\
count	54.000000	54.000000	54.000000	
mean	8.098148	3.592593	3.407407	
min	4.500000	2.000000	1.000000	
25%	6.000000	3.000000	3.000000	
50%	7.000000	4.000000	3.000000	
75%	8.000000	4.000000	4.000000	
max	45.000000	5.000000	5.000000	
std	5.725464	0.981307	0.835950	

	package_expectation_lpa	cia_pct	gpa	attendance_pct
count	54.000000	54.000000	54.000000	54.000000
mean	13.629630	73.88963	3.476800	93.163519

min	5.000000	60.00000	2.900000	85.000000
25%	7.000000	70.00000	3.377500	90.000000
50%	8.500000	71.11000	3.478400	93.500000
75%	11.500000	78.99750	3.600000	97.000000
max	200.000000	89.00000	4.000000	100.000000
std	26.876936	6.42520	0.258752	4.528431

9. Select appropriate dependent and independent variables

For this analysis, **GPA** is selected as the dependent variable since it represents the academic outcome to be predicted. **CIA percentage** and **Attendance percentage** are treated as independent variables because they are expected to influence a student's academic performance and may serve as predictors of GPA in the regression models.

1.2 Part B: Simple Linear Regression using Scikit-learn

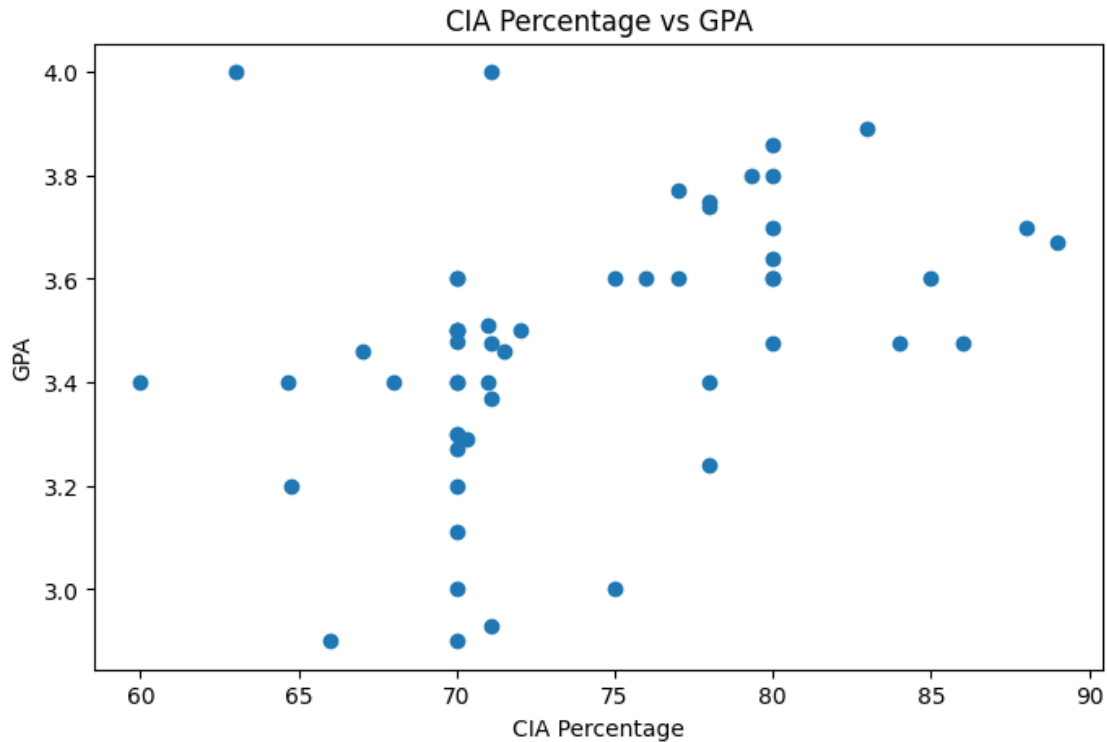
1.2.1 Regression Experiments: Experiment 1

Independent Variable (X): CIA Percentage

Dependent Variable (Y): GPA

```
[154]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.scatter(df['cia_pct'], df['gpa'])
plt.xlabel('CIA Percentage')
plt.ylabel('GPA')
plt.title('CIA Percentage vs GPA')
plt.show()
```



```
[155]: correlation = df['cia_pct'].corr(df['gpa'])

print(f"Correlation Coefficient: {correlation:.4f}")
```

Correlation Coefficient: 0.4111

Moderate Correlation between CIA% and GPA

```
[156]: X = df[['cia_pct']]
y = df['gpa']
```

```
[157]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X,y,test_size=0.
↪2,random_state=42)
```

```
[158]: from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

```
[159]: model.fit(X_train, y_train)
```

```
[159]: LinearRegression()
```

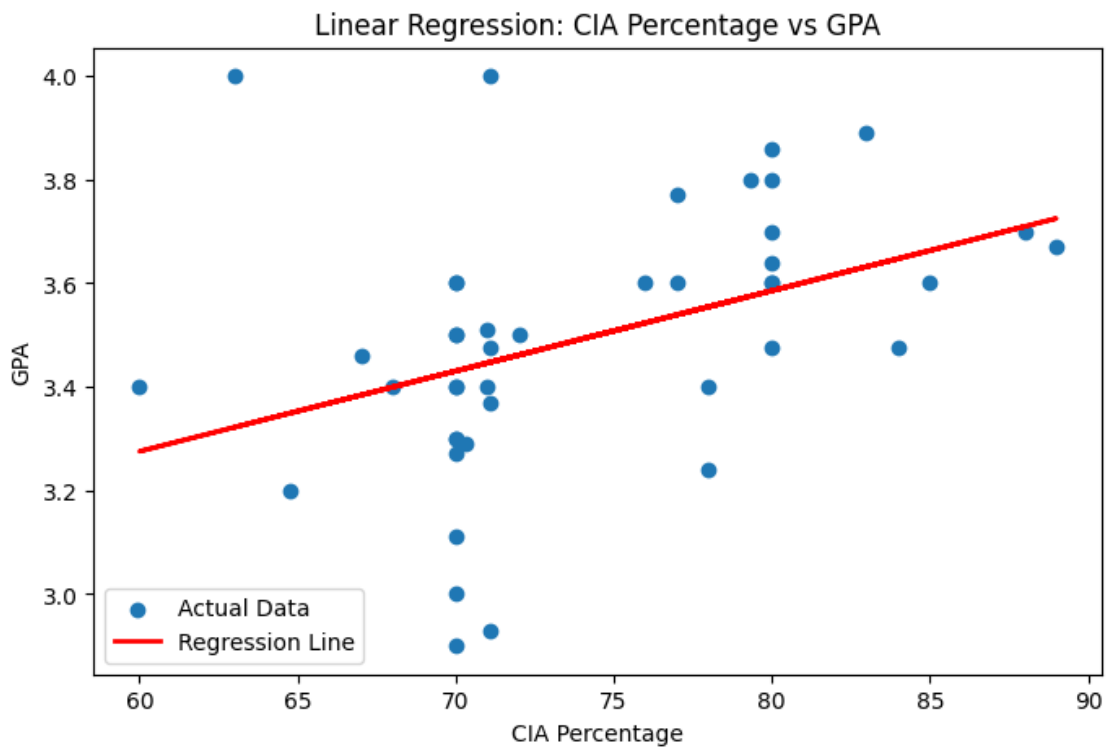
```
[160]: plt.figure(figsize=(8,5))
```



```
plt.scatter(X_train, y_train, label='Actual Data')

plt.plot(
    X_train,
    model.predict(X_train),
    color='red',
    linewidth=2,
    label='Regression Line'
)

plt.xlabel('CIA Percentage')
plt.ylabel('GPA')
plt.title('Linear Regression: CIA Percentage vs GPA')
plt.legend()
plt.show()
```



```
[161]: slope = model.coef_[0]
slope
```

```
[161]: np.float64(0.01550953180679792)
```

```
[162]: intercept = model.intercept_
intercept
```

```
[162]: np.float64(2.344861579132017)
```

$$\text{GPA} = 2.3449 + 0.0155 \times (\text{CIA Percentage})$$

```
[163]: y_pred = model.predict(X_test)
```

```
[164]: results = pd.DataFrame({'Actual GPA': y_test, 'Predicted GPA': y_pred})
results
```

```
[164]:
```

	Actual GPA	Predicted GPA
19	3.7500	3.554605
49	3.7400	3.554605
48	3.4600	3.453793
12	3.4800	3.430529
44	3.0000	3.508076
5	3.5000	3.430529
17	2.9000	3.368491
52	3.4768	3.678681
3	3.2000	3.430529
32	3.6000	3.508076
13	3.4000	3.347863

```
[165]: from sklearn.metrics import mean_squared_error, r2_score
```

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE: {mse:.4f}")
print(f"R2 Score: {r2:.4f}")
```

MSE: 0.0602

R² Score: 0.1201

The Simple Linear Regression model produced an MSE of 0.0602 and an R² score of 0.1201. This indicates that CIA percentage explains approximately 12.01% of the variation in GPA. While a relationship exists between CIA percentage and GPA, the low R² value suggests that CIA percentage alone is not a strong predictor of GPA, and additional factors may influence student academic performance.

```
[166]: plt.figure(figsize=(8,5))

plt.scatter(y_test, y_pred)

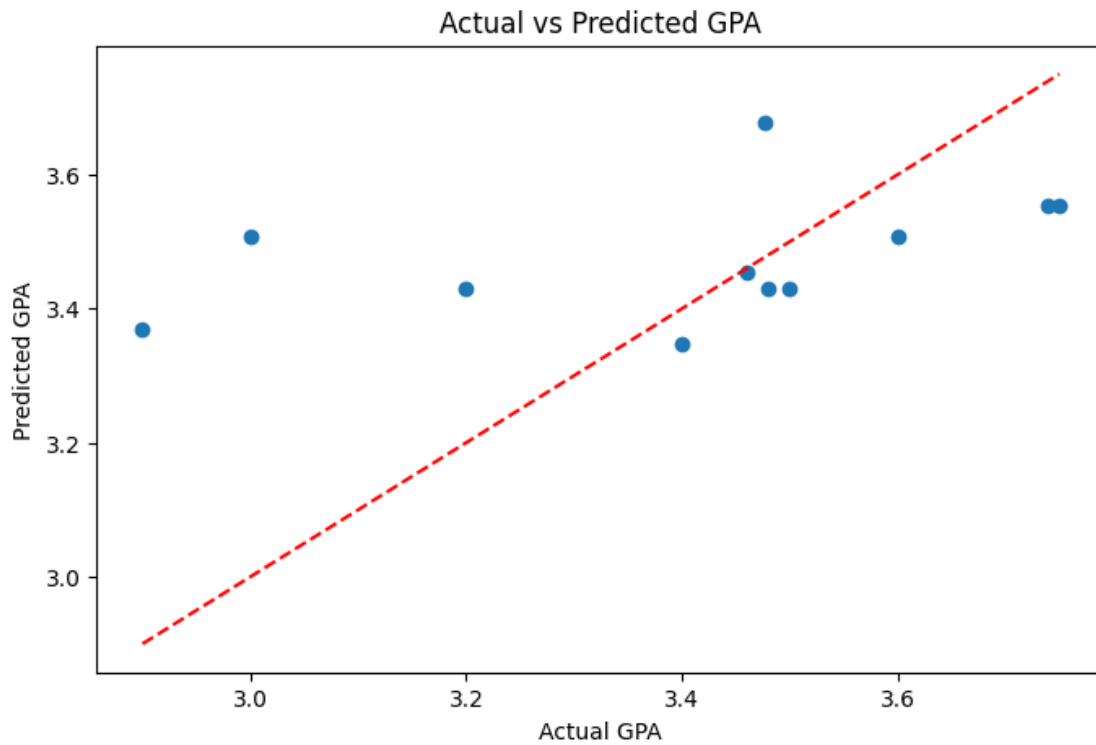
plt.xlabel("Actual GPA")
plt.ylabel("Predicted GPA")
plt.title("Actual vs Predicted GPA")

plt.plot(
    [y_test.min(), y_test.max()],
```

```

    [y_test.min(), y_test.max()],
    'r--'
)
plt.show()

```



1.2.2 Regression Experiments: Experiment 2

Independent Variable (X): Attendance Percentage

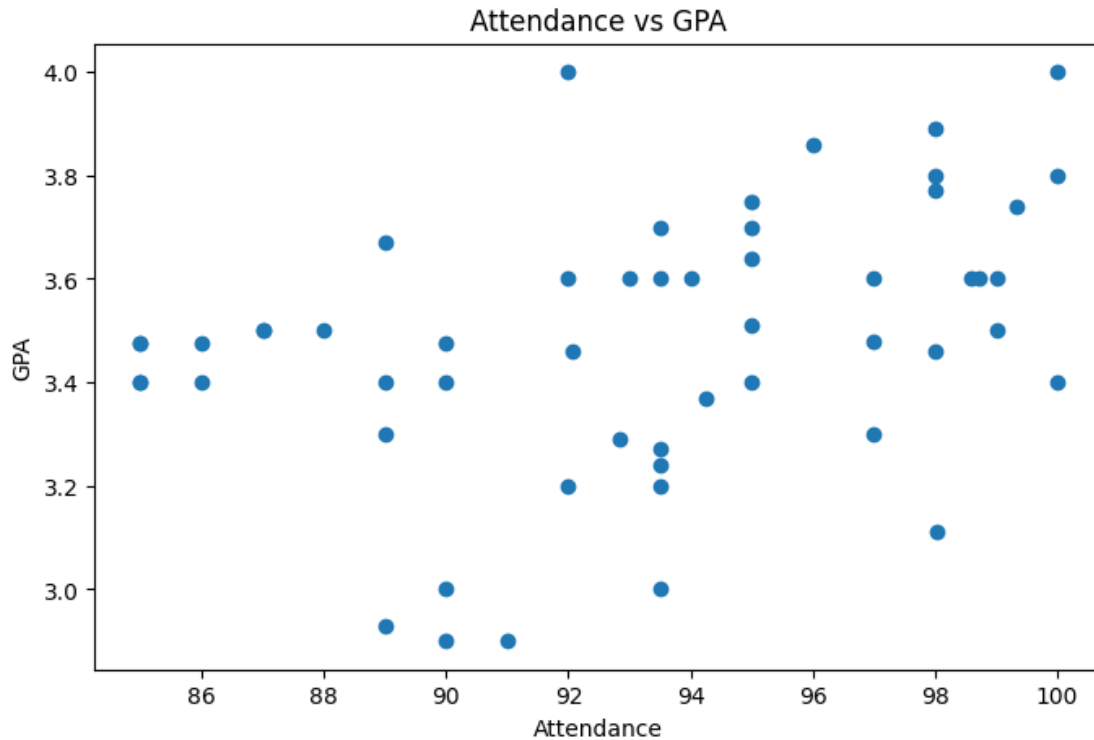
Dependent Variable (Y): GPA

```

[167]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.scatter(df['attendance_pct'], df['gpa'])
plt.xlabel('Attendance')
plt.ylabel('GPA')
plt.title('Attendance vs GPA')
plt.show()

```



```
[168]: correlation = df['attendance_pct'].corr(df['gpa'])
print(f"Correlation Coefficient: {correlation:.4f}")
```

Correlation Coefficient: 0.3593

Moderate Correlation

```
[169]: X = df[['attendance_pct']]
y = df['gpa']
```

```
[170]: X_train, X_test, y_train, y_test = train_test_split( X,y,test_size=0.
↪2,random_state=42)
```

```
[171]: model = LinearRegression()
```

```
[172]: model.fit(X_train, y_train)
```

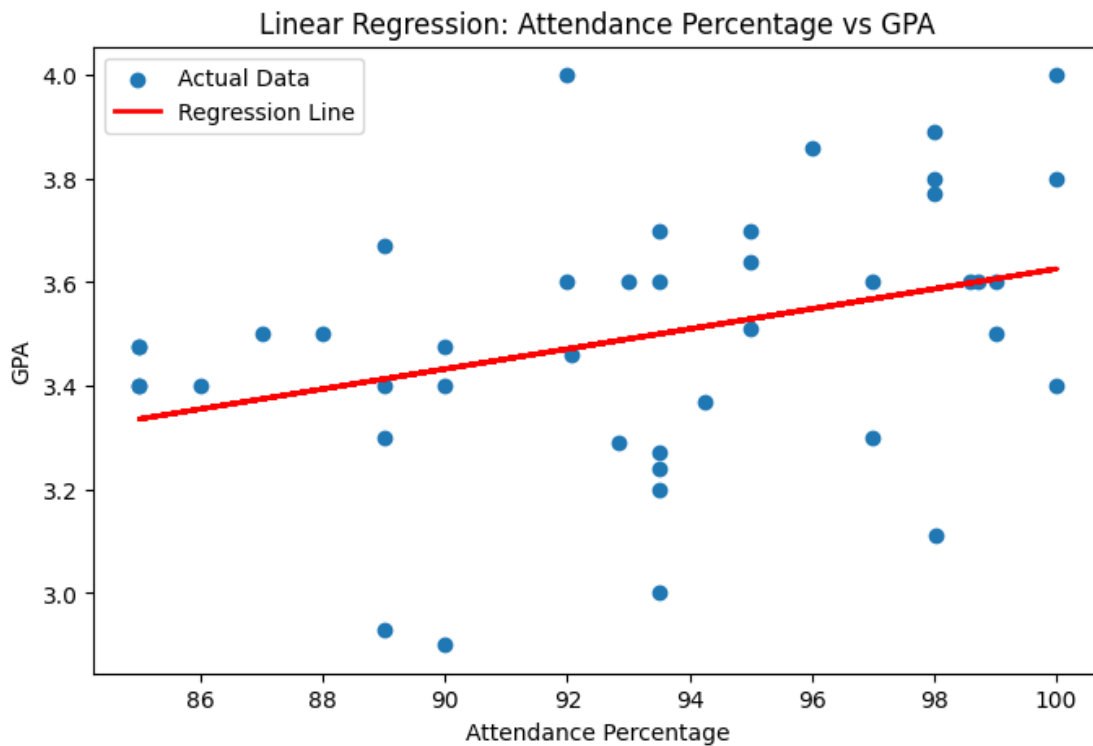
```
[172]: LinearRegression()
```

```
[173]: plt.figure(figsize=(8,5))

plt.scatter(X_train, y_train, label='Actual Data')
```

```
plt.plot(
    X_train,
    model.predict(X_train),
    color='red',
    linewidth=2,
    label='Regression Line'
)

plt.xlabel('Attendance Percentage')
plt.ylabel('GPA')
plt.title('Linear Regression: Attendance Percentage vs GPA')
plt.legend()
plt.show()
```



```
[174]: slope= model.coef_[0]
slope
```

```
[174]: np.float64(0.019322402070930896)
```

```
[175]: intercept = model.intercept_
intercept
```

```
[175]: np.float64(1.6936137052752585)
```

$$\text{GPA} = 0.019(\text{Attendance } \%) + 1.69$$

```
[176]: y_pred = model.predict(X_test)
```

```
[177]: results = pd.DataFrame({'Actual GPA': y_test, 'Predicted GPA': y_pred})
results
```

```
[177]:
```

	Actual GPA	Predicted GPA
19	3.7500	3.529242
49	3.7400	3.613101
48	3.4600	3.587209
12	3.4800	3.567887
44	3.0000	3.432630
5	3.5000	3.374663
17	2.9000	3.451952
52	3.4768	3.355340
3	3.2000	3.471275
32	3.6000	3.509919
13	3.4000	3.529242

```
[178]: mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE: {mse:.4f}")
print(f"R2 Score: {r2:.4f}")
```

MSE: 0.0645

R² Score: 0.0578

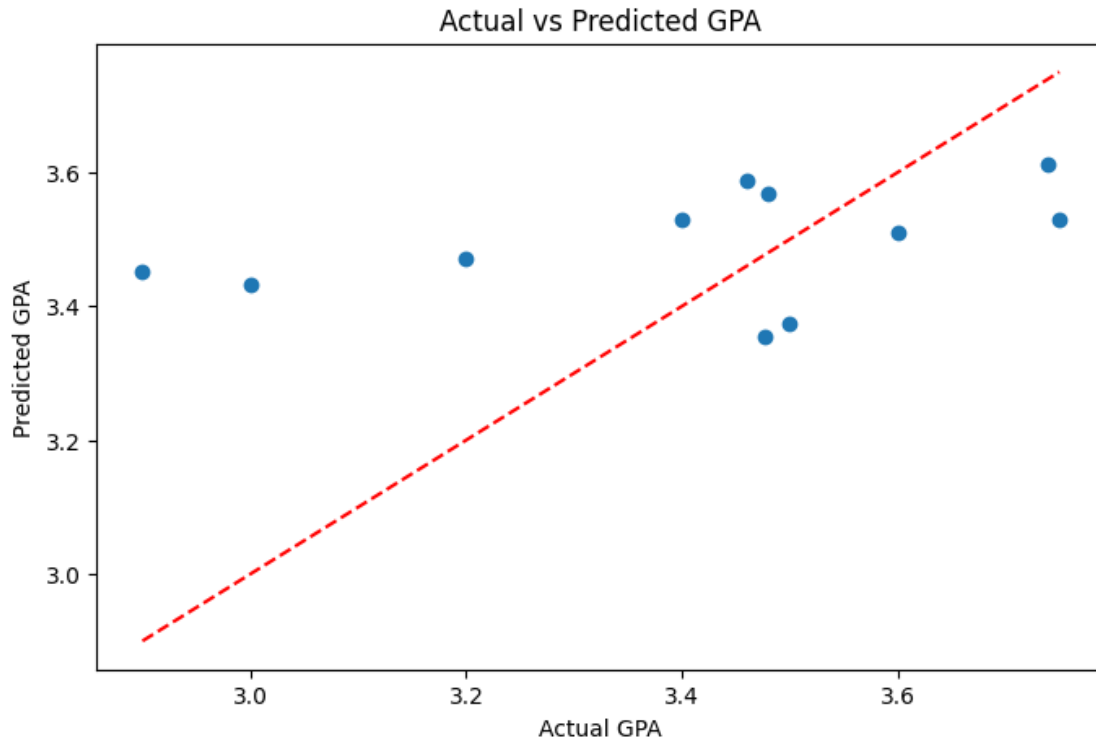
```
[179]: plt.figure(figsize=(8,5))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual GPA")
plt.ylabel("Predicted GPA")
plt.title("Actual vs Predicted GPA")

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)

plt.show()
```



1.3 Part C: Manual Computation using Ordinary Least Squares (OLS)

```
[180]: X = df[['attendance_pct']]
y = df['gpa']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42)
```

Arrays initialized for manual OLS computation.

```
[181]: X_train_arr = X_train['attendance_pct'].to_numpy()
y_train_arr = y_train.to_numpy()
```

```
[182]: X_mean = np.mean(X_train_arr)
y_mean = np.mean(y_train_arr)

numerator = np.sum((X_train_arr - X_mean) * (y_train_arr - y_mean))
denominator = np.sum((X_train_arr - X_mean) ** 2)

manual_slope = numerator / denominator
manual_intercept = y_mean - (manual_slope * X_mean)

print("Manual OLS Parameters")
print(f"Manual Slope: {manual_slope}")
```

```
print(f"Manual Intercept: {manual_intercept}")
print(f" Equation: GPA = {manual_intercept:.4f} + {manual_slope:.4f} *_
↪(attendance_pct)")
```

Manual OLS Parameters

Manual Slope: 0.0193224020709309

Manual Intercept: 1.693613705275258

Equation: GPA = 1.6936 + 0.0193 * (attendance_pct)

```
[183]: X_test_arr = X_test['attendance_pct'].to_numpy()
manual_predictions = manual_intercept + (manual_slope * X_test_arr)
```

```
[ ]: difference = y_pred - manual_predictions

comparison_df = pd.DataFrame({
    'Actual GPA': y_test,
    'Scikit-learn Prediction': y_pred,
    'Manual OLS Prediction': manual_predictions,
    'Difference': difference
})

print("Prediction Comparison:")
print(comparison_df.head(10)) # Shows the first 10 rows
```

```
=== Prediction Comparison ===
```

	Actual GPA	Scikit-learn Prediction	Manual OLS Prediction	Difference
19	3.7500	3.529242	3.529242	4.440892e-16
49	3.7400	3.613101	3.613101	0.000000e+00
48	3.4600	3.587209	3.587209	0.000000e+00
12	3.4800	3.567887	3.567887	0.000000e+00
44	3.0000	3.432630	3.432630	0.000000e+00
5	3.5000	3.374663	3.374663	0.000000e+00
17	2.9000	3.451952	3.451952	4.440892e-16
52	3.4768	3.355340	3.355340	0.000000e+00
3	3.2000	3.471275	3.471275	0.000000e+00
32	3.6000	3.509919	3.509919	0.000000e+00

```
[ ]: import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.scatter(X_test_arr, y_test, color='gray', alpha=0.6, label='Actual Test_
↪Data Points')

plt.plot(X_test_arr, y_pred, color='blue', linewidth=3, linestyle='-',_
↪label='Scikit-learn Line')

plt.plot(X_test_arr, manual_predictions, color='red', linewidth=1.5,_
↪linestyle='--', label='Manual OLS Line')
```



```
plt.title('Part C Verification: Scikit-learn Line vs. Manual OLS Line',
         ↪fontsize=14, fontweight='bold')
plt.xlabel('Attendance Percentage (attendance_pct)', fontsize=12)
plt.ylabel('GPA', fontsize=12)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(fontsize=11)

plt.show()
```

